



PDF Download
3731569.3764797.pdf
29 March 2026
Total Citations: 0
Total Downloads: 3774



Published: 12 October 2025

Citation in BibTeX format

SOSP '25: ACM SIGOPS 31st Symposium
on Operating Systems Principles
October 13 - 16, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3731569.3764797>

RESEARCH-ARTICLE

eBPF Misbehavior Detection: Fuzzing with a Specification-Based Oracle

TAO LYU, EPFL, Lausanne, Switzerland

KUMAR KARTIKEYA DWIVEDI, EPFL, Lausanne, Switzerland

THOMAS BOURGEAT, EPFL, Lausanne, Switzerland

MATHIAS PAYER, EPFL, Lausanne, Switzerland

MENG XU, University of Waterloo, Waterloo, ON, Canada

SANIDHYA KASHYAP, EPFL, Lausanne, Switzerland

Open Access Support provided by:

EPFL

University of Waterloo



eBPF Misbehavior Detection: Fuzzing with a Specification-Based Oracle

Tao Lyu Kumar Kartikeya Dwivedi

Thomas Bourgeat Mathias Payer Meng Xu[†] Sanidhya Kashyap

EPFL [†]University of Waterloo

Abstract

Bugs in the Linux eBPF verifier may cause it to mistakenly accept unsafe eBPF programs or reject safe ones, causing either security or usability issues. While prior works on fuzzing the eBPF verifier have been effective, their bug oracles only hint at the existence of bugs *indirectly* (e.g., when a memory error occurs in downstream execution) instead of showing the root cause, confining them to uncover a narrow range of security bugs only with no detection of usability issues.

In this paper, we propose SPECHECK, a *specification-based oracle* integrated with our fuzzer VERITAS, to detect a wide range of bugs in the eBPF verifier. SPECHECK encodes eBPF instruction semantics and safety properties as a specification and turns the claim of whether a concrete eBPF program is safe into checking the satisfiability of the corresponding safety constraints, which can be reasoned automatically without abstraction. The output from the oracle will be cross-checked with the eBPF verifier for any discrepancies. Using SPECHECK, VERITAS uncovered 13 bugs in the Linux eBPF verifier, including severe bugs that can cause privilege escalation or information leakage, as well as bugs that cause frustration in even experienced kernel developers.

1 Introduction

Kernel extensions are critical components of modern operating systems that allow developers to extend the kernel with custom functionality. eBPF is one such framework that allows developers to extend the Linux kernel [15] safely. eBPF, at its core, relies on static verification to ensure the safety of eBPF-based extensions (*i.e.*, eBPF programs). The eBPF verifier validates every program before execution, thereby preventing unsafe operations that could compromise the kernel.

This role becomes even more critical with the widespread industrial deployment of eBPF-based extensions. For instance, servers at Meta each execute over 50 eBPF programs [21], underscoring the extensive reliance on these extensions in large-scale deployments. To ensure the safety of such programs, the Linux eBPF verifier statically checks each eBPF program against a set of safety conditions the kernel expects (e.g., in-bounds memory access) using a heuristic algorithm that combines domain abstraction [22] and execution path enumeration.

Unfortunately, the eBPF verifier contains bugs that either *reject safe eBPF programs (usability issues)* or *accept unsafe ones (security issues)*. Specifically, rejecting safe eBPF programs imposes significant overhead on developers, who must spend time debugging the correct code while struggling to understand the subtleties in the verifier [3]; or, accepting unsafe eBPF programs imposes security risks, such as privilege escalation [6] and information leakage [7].

As the bugs in the verifier can lead to critical issues, we need a deeper understanding of the current verifier design to develop effective approaches for detecting and fixing bugs. We identify four primary root causes of eBPF verifier bugs. First, abstract interpretation in the eBPF verifier is imprecise—over-approximating eBPF program states—hence, rejecting safe programs (RC1). Second, safety checks, implemented incrementally with new features (e.g., new instructions or kernel functions) by different developers can be inconsistent, either too conservative or too relaxed (RC2). Furthermore, even with the required informal documentation of safety properties—actually absent, developers still make implementation mistakes (RC3), especially in optimization heuristics (RC4). Overall, RC1–3 can cause safe eBPF programs to be rejected, while RC3 and RC4 allow accepting the unsafe eBPF programs, thereby raising critical concerns.

Existing works have focused on finding a *subset* of bugs in RC3 and RC4, with less attention on RC1 and RC2. For instance, existing formal verification works specifically verify the functional correctness of a single component. Agni [39], for example, targets the range analysis component to rule out range-specific bugs (RC3). Automated testing, on the other hand, relies on specific runtime patterns, such as memory sanitizer KASAN [27, 37] or integer range discrepancies between the verifier’s approximated states and eBPF program runtime states [26, 36], to detect issues in RC3 and RC4.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. SOSP '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1870-0/2025/10

<https://doi.org/10.1145/3731569.3764797>

In this paper, we take a rather holistic approach to nip the identified root causes in their bud. We introduce the fuzzing framework VERITAS, which utilizes the specification-based oracle SPECHECK that identifies eBPF verifier bugs based on the aforementioned root causes (RC1–RC4). Specifically, SPECHECK systematically specifies eBPF instruction semantics (especially the dynamic type system) and five safety properties of eBPF programs guaranteed by the eBPF VM. The specification is further encoded as pre- and post-conditions of each instruction using axiomatic notations, which is *extensible* as adding new instructions or kernel functions is simply to extend the specification with the new pre- and post-conditions. Moreover, SPECHECK leverages the specification to verify the safety of an eBPF program through SMT solvers, ensuring the *trackability* of failed conditions and the oracle as *precise* as the SMT solvers.

VERITAS found 15 new bugs in total, indicating its bug-finding effectiveness. Some bugs can cause severe security consequences, such as privilege escalation to root and information leaks capable of bypassing KASLR, or usability issues that cost eBPF developers hours to debug. 12 bugs have been acknowledged, and eight of them have already been fixed.

Summary. This paper makes the following contributions:

- **Specification of eBPF instruction semantics and safety properties:** We systematically analyzed the eBPF VM to specify its instruction semantics, particularly its type system, and the safety properties it guarantees. This provides the community with a deeper understanding of the eBPF VM, facilitating the further development of the eBPF-based extension system.
- **A specification-based oracle for holistic detection of eBPF verifier bugs:** We present the specification-based checker SPECHECK and integrate it into our fuzzer VERITAS, enabling the automatic and comprehensive detection of various types of bugs in the eBPF verifier.
- **Finding severe bugs:** VERITAS uncovered bugs that can lead to severe security vulnerabilities, such as privilege escalation and information leaks, as well as significant usability issues for developers.

VERITAS is publicly available at <https://github.com/rs3lab/veritas>.

2 Background and Motivation

We introduce the basics of the eBPF virtual machine (VM) and illustrate four Linux eBPF verifier bugs uncovered by VERITAS to highlight the limitations of existing works.

2.1 eBPF Virtual Machine and Verifier

Basics. The eBPF VM is a register-based architecture with its dedicated eBPF Instruction Set Architecture (ISA) [9], designed to execute kernel extensions for system tracing,

security enforcement, and network processing. It has ten general-purpose 64-bit registers (R0–R9) and one read-only stack frame pointer register (R10), pointing to a fixed-size stack memory region private to each eBPF VM, similar to the rbp register on x86-64. The eBPF VM also provides other types of special-purpose memory regions to interact with the kernel and userspace. Programs running inside the eBPF VM include an entry function (*i.e.*, main function) and optionally, auxiliary functions (*i.e.*, pseudo functions). Each function is a finite sequence of eBPF instructions [9].

The Linux eBPF verifier. The goal of the Linux eBPF verifier is to ensure that an untrusted eBPF program is “safe” to execute in kernel space. While we defer the discussion of safety requirements to §3.3, in a nutshell, an eBPF program cannot access arbitrary kernel data structures nor cause hangs, panics, or resource leaks.

To achieve this, the eBPF verifier implements an algorithm that preserves some safety properties. The algorithm comprises some form of abstract interpretation [22] with execution path enumeration commonly found in symbolic execution (*e.g.*, KLEE [19]). Specifically, it first performs control flow graph validation to preclude infinite loops and recursion, and subsequently enumerates all program paths. For each path, the verifier simulates the execution of every instruction, tracks the state change of registers and memory regions, and checks that even over-approximated execution states (*e.g.*, integers approximated with wider possible ranges) are still safe. Moreover, it prunes execution paths heuristically to reduce path explosion and improve verification efficiency.

Upon passing verification, eBPF programs are attached to specific kernel hooks to be executed either by the interpreter or as JIT-compiled machine code. In both cases, the safety checks done by the static verifier are not repeated at runtime.

2.2 Issues with the Linux eBPF Verifier

We now discuss the four root causes (RC) that the current verifier suffers from.

```

1  uint32_t array[11];
2  // x is aligned to 4 bytes
3  // x is initialized with a value loaded from a context field
4  uint32_t x = ctx_value;
5  uint32_t res;
6  // Bound the range of x
7  if (x >= 0 && x < 11) {
8      res = compute_value();
9      array[x] = res; // Rejected
10 }
```

Figure 1. A bug caused by imprecise abstract interpretation and found by VERITAS took a sched-ext developer hours to debug.

RC1: Imprecision caused by state abstraction. Even in the best-case scenario, where all contributors to the Linux eBPF verifier have a shared and complete understanding of safety properties, and the implementation is flawless, the eBPF verifier can still reject safe eBPF programs due to the inherent conservative approximation of execution states.

Figure 1 shows such a case discovered by VERITAS. The example program declares a 44-byte local array (array) and a 32-bit variable x, which is at a 4-byte aligned address and initialized to an unknown value read from a field in a context memory region. The range of x is then bounded to safely index into array (line 7). Subsequently, array[x] is assigned the return value of a pseudo function compute_value (line 9). However, the verifier rejects this program, incorrectly determining that x could exceed the array bounds despite the earlier bounds check.

The root cause lies in the “intended” abstract interpretation design of not propagating the numerical range state from registers to memory regions that are not 8-byte aligned. Therefore, when x is loaded from memory into a register r1 and undergoes a bounds check (line 7), the verifier does not propagate the range information to the 4-byte aligned stack data. Later, r1 and its maintained range information is overwritten when computing res. When x is subsequently reloaded into register (line 9), the verifier has no record of its previously established bounds, causing it to conservatively assume x could be out of bounds and incorrectly rejecting the program. This creates a bad experience as reported by frustrated sched-ext [12] developers after our discovery.

```
1 int *ptr1 = ptr;
2 atomic_and(&ptr1, 1); // Success: & between 1 and ptr in &ptr1
3 ptr &= 1;           // Failed : & between 1 and ptr
```

Figure 2. Inconsistent safety constraints in bitwise operation

RC2: inconsistent safety rules. Ideally, participants in the eBPF ecosystem should share the same and complete understanding of safety properties. However, in reality, such systematic knowledge is nonexistent. The safety of eBPF programs is more like folklore knowledge imprecisely described in natural languages (e.g., mailing lists[4], code comments [11] and selftests [13]), and safety checks in the verifier are often implemented incrementally as the eBPF ISA expands (e.g., new instructions and kernel functions). As different developers offer individual and uncoordinated understanding of safety rules, inconsistent safety checks—checks that are more conservative or relaxed than similar counterparts without a clear justification—are not uncommon.

As shown in Figure 2, a regular bitwise AND and an atomic AND on a pointer yield inconsistent verification results, confusing eBPF application developers. Since both operations pose the same security risk (potential pointer leakage, see §3.3) with no additional safety concerns, they should be treated consistently. For privileged users who are allowed to leak pointers, both variants should be permitted to maximize programming flexibility.

RC3: incorrect implementation. Besides inconsistent safety notions, the implementation is rarely perfect. Developers might implement an incomplete or even wrong set of safety rules, which can cause either safe eBPF programs

```
1 struct bpf_iter_num it;
2 // memory data pointing by map_val is controlled by users.
3 int *map_val = ...;
4
5 bpf_iter_num_new(&it, 0, 3);
6 // Correct one: while (bpf_iter_num_next(&it)) {}
7 while (bpf_iter_num_next((struct bpf_iter_num *)map_val)) {}
8 bpf_iter_num_destroy(&it);
```

Figure 3. A bug that misses type checks found by VERITAS.

being rejected or unsafe eBPF programs being accepted, due to missing specifications for required safety rules.

Figure 3 illustrates a bug where the verifier fails to type check properly. The program declares a number enumeration structure with start and end fields defining a range [0-3). The program then calls the kernel function bpf_iter_num_next in a loop to iterate over the range. bpf_iter_num_next expects the enumeration structure to reside on the stack; allocating memory dynamically within the function to maintain it would degrade performance in latency-critical eBPF contexts. Internally, bpf_iter_num_next takes the enumeration structure, returns the current start value, and increments the start value by 1. It returns null when start equals end. However, due to missing type checks in the verifier, the program can pass any arbitrary memory pointer (e.g., a map pointer) as the argument instead of a valid enumeration structure. For example, a user can modify data in mapped memory from userspace during runtime, potentially causing the eBPF program to loop infinitely, leading to a hang in kernel space.

```
1 int func1() {
2     uint64_t *victim_ptr;
3     // ptr @(r10-8) saves its own address
4     victim_ptr = (uint64_t *)&victim_ptr;
5     return 0;
6 }
7 int func2() {
8     uint64_t leaked_ptr; // Not initialized
9     ((char *)&leaked_ptr)[0] = 0;
10    // Bug: leaked_ptr becomes a readable 8-byte integer
11    // but its high 7 bytes are partial victim_ptr
12    return 0;
13 }
14 SEC("socket")
15 int leak_ptr(void *ctx) { func1(); func2(); }
```

Figure 4. An erroneous optimization leads to data leakage.

RC4: erroneous optimization. Historically, a hot spot of implementation errors is the optimizations in the verifier as they increase the complexity of the verification algorithm significantly. Hence, we assign a special tag to bugs caused by optimizations and differentiate them from RC3. Specifically, to improve verification efficiency, the verifier employs path pruning via path equivalence checks. On encountering a branch starting at instruction i , the verifier checks if it has already verified this branch with a previous program state S_1 . If the current state S_2 can be subsumed by S_1 the verifier skips re-verifying that branch. Essentially, the verifier only attempts to unify register values, stack data, and other states used in memory access instructions from S_2 with S_1 .

VERITAS discovered a bug in path pruning. The verifier implements a special behavior for programs with capability `CAP_PERFMON`, which allows leaking kernel data to userspace. When spilling an N -byte value to an uninitialized 8-byte aligned stack address, it marks the remaining $(8 - N)$ bytes as known values. This allows more states to be unified for pruning purposes. The bug arises when the verifier mistakenly applies this behavior to programs without `CAP_PERFMON`. This can be exploited to leak kernel pointers, potentially bypassing KASLR[2] and enabling more severe attacks.

Figure 4 demonstrates an exploit for this bug. The entry function `leak_ptr` (line 15) first calls `func1`, which saves a stack address in `victim_ptr` at the top of the stack frame. It then calls `func2`, which shares the same stack frame and writes a single byte to `leaked_ptr`—a location residing in the same eight-byte stack slot as `victim_ptr`. Due to the verifier bug, although only one byte is written, the verifier incorrectly marks the next 7 bytes as initialized integers. These 7 bytes actually contain part of the pointer value stored in `victim_ptr`. The program can then load these bytes as integers and leak them to userspace.

2.3 Improving Assurance of eBPF Verifier

Given the importance of the Linux eBPF verifier, several works try to improve its correctness, which can be broadly categorized into three themes:

Formal verification aims to prove that the verifier conforms to a specific set of correctness specifications. For example, several works [38, 39] proved the correctness of range analysis. Despite its high assurance, formal verification often struggles to scale due to the verifier’s large (20K LoC) and rapidly evolving codebase. This may explain the lack of full-spectrum verification beyond range analysis.

Alternative designs with solid language-theoretic foundation also exist. For instance, PREVAIL [24] seeks to capture and unify ad-hoc safety rules in eBPF verifier and reimplement them with a foundational framework such as abstract interpretation. This means the alternative designs are typically more accurate with reasonable or even no performance penalties. The major drawback, however, is that it will be challenging for the alternative verifier to keep up with the rapid evolution of the eBPF verifier.

Fuzzing [40] as one of the most effective bug-finding approaches has been applied on eBPF verifier [26, 27, 36, 37]. While coverage-guided fuzzing can be very effective in exploring different parts of the verifier, a fuzzer still needs a bug oracle to decide when the eBPF verifier is erroneous, and the design of an eBPF-specific bug oracle has been a differentiating factor in prior works.

Bug oracles in prior works include memory sanitizer KASAN [27, 37] and a reference monitor for scalar range inconsistencies between verifier states and runtime states [26, 36]. These oracles, however, can detect only a subset of bugs

caused by RC3 and RC4 as many runtime errors (e.g., information leaks or type confusion) do not necessarily lead to memory errors. These oracles also completely forgo the chance of finding issues in RC1 and RC2 as eBPF programs that fail verification will not even be executed.

2.4 Motivation and Insight

The unique characteristics of each theme drive us to consider a holistic yet practical scheme to find all bug types listed in §2.2. We present VERITAS, a fuzzing-based testing framework with an oracle SPECHECK, built on an extensible set of specifications that encode constraints for eBPF programs. SPECHECK employs automated reasoning to verify eBPF programs against these specifications. If the verification result diverges from the eBPF verifier’s result, SPECHECK flags it as a bug. Importantly, we do not propose SPECHECK as a replacement for the in-production eBPF verifier. Rather, SPECHECK serves as a precise and extensible testing oracle at the cost of verification speed.

We first present the design of the specification-based oracle (§3), SPECHECK, which we later integrate into VERITAS (§4).

3 SPECHECK: A Specification-based Oracle

VERITAS aims to identify eBPF verifier bugs holistically through a specification-based oracle SPECHECK, designed with the following four goals in mind:

Goal 1: Precision. SPECHECK should accurately characterize the set of safe eBPF programs by precisely specifying program state and safety constraints. Any imprecision could lead to false alarms when comparing results with the Linux eBPF verifier, making it difficult to determine if the verifier is functioning correctly.

Goal 2: Trackability. When facing an unsafe program, SPECHECK must be able to pinpoint the root cause of safety violations. That is, SPECHECK can blame the specific instruction that violated the expected eBPF safety guarantees.

Goal 3: Extensibility. The eBPF ecosystem continuously evolves with new instructions and helper functions. SPECHECK should be easily extended to model the new features, enabling thorough testing of both new, existing components, and their interplay in the eBPF verifier, without requiring significant changes to the framework.

Goal 4: Reasonability. The specification underlying SPECHECK must be amenable to formal reasoning, enabling us to prove higher-level properties (e.g., in the form of lemmas/theorems) that the specification should enforce.

3.1 Overview of SPECHECK

To achieve these goals, SPECHECK is built around two components: 1) The operational semantics and constraints on alignments and operations that the eBPF specification mandates for each eBPF instruction [9] (including kernel functions). We augment this operational semantics with dynamic

types and the corresponding typing rules to track the type of values manipulated by the eBPF VM. We aim for precision by avoiding abstraction where possible (§3.2). 2) A set of safety constraints for each eBPF instruction derived systematically to ensure five safety properties (§3.3).

These components are specified using per-instruction and pure functions (§3.4). This approach directly supports *extensibility*, as incorporating a new instruction only requires adding its corresponding pre- and post-conditions, and *reasonability*, as the use of pure functions facilitates reasoning.

More importantly, these specifications enable SPECHECK to turn the question of “*Is a given eBPF program safe?*” into proof obligations that can be discharged to automated theorem provers, *e.g.*, satisfiability modulo theories (SMT) solvers. By encoding all dynamic checks performed in the semantics into SMT formulae, SPECHECK avoids any form of approximation, achieving a high degree of *precision*, bounded primarily by the capabilities of the underlying solver to solve the verification obligations.

Furthermore, when SPECHECK fails an eBPF program verification, the SMT solver’s counterexample can be used by SPECHECK to identify the specific constraints that were violated, directly providing *trackability*.

3.2 eBPF Semantic Specification

SPECHECK encodes the semantics of eBPF instructions with their operational semantics and constraints as defined in the eBPF ISA [9]. The semantics is augmented with a dynamic type system, characterizing how types are updated by each instruction. We provide detailed definitions of terms and dynamic typing rules in the appendix.

Terms. An eBPF program consists of a sequence of instructions (see Figure 12 in the appendix)—categorized as arithmetic, data handling, memory, and control flow operations—each of which takes immediates or registers as operands.

Types. SPECHECK defines a dynamic type system that associates a data type with its value, as shown below.

```
datatype ETYPEV =
  | Uninit
  | Scalar(kind: Kinds, val: bv64)
  | PtrType(r: MemRegion, memid: nat, off: bv64)
  | PtrOrNullType(r: MemRegion, memid: nat, off: bv64)
```

Uninit denotes uninitialized data. After initialization, data can exist in one of three forms: a `Scalar(kind, val)` representing an integer value (1 to 8 bytes) annotated with its semantic kind; a non-null pointer `PtrType(r, memid, off)`; or a potentially null pointer `PtrOrNullType(r, memid, off)`. As memory in the eBPF VM is discontinuous, we model it as multiple memory regions. Each region is uniquely identified by a triple (`r`, `memid`, `off`), where `r` denotes the region type (*e.g.*, stack or

packet), `memid` identifies the specific region within that type, and `off` specifies the byte-level offset within the region.

Model. SPECHECK models the eBPF VM as a state machine, in which an eBPF program is comprised of functions, each containing a sequence of instructions that operate on registers and separate memory regions. SPECHECK tracks the state (*i.e.*, value and type) of registers and memory bytes, along with memory access permissions (*e.g.*, no access, read-only, and read-write) and other states (*e.g.*, the meta information of maps), forming the execution context.

Registers `r0`-`r9` and the stack memory region are general-purpose, capable of storing any data type. In contrast, register `r10` and other memory regions are special-purpose and fixed to specific and unchanging types. These memory regions are either *raw* memory, where all data is of type scalar, or *structured* memory with fields of type scalar or pointer. For instance, context memory containing socket buffer data is structured with the `__sk_buff` kernel structure. Notably, since pointers are eight bytes, a pointer stored in memory indicates that all 8 bytes starting at the address must be marked with `PtrType` or `PtrOrNullType`.

Initial context. Initially, `r1` and `r10` represent the base address of the context and stack memory, respectively. Thus, their typed values are set to `PtrType(stack, 0, 0)` and `PtrType(context, 0, 0)`. Other registers and memory locations are initialized as follows: Registers and stack slots are all initialized to `uninit`, while other memory bytes are initialized with their predefined types—either scalar, `PtrType`, or `PtrOrNullType`—based on their intended purpose.

We now describe SPECHECK’s type rules, key instruction semantics, and semantic constraints, which every eBPF program must satisfy.

Arithmetic instructions. These operations fall into two categories: single operand (*e.g.*, bitwise negation) and double operand (*e.g.*, addition). The semantics of arithmetic instructions align with those of other ISAs, except for special cases like division-by-zero or modulo-by-zero. For example, division-by-zero is allowed and returns zero.

In SPECHECK’s dynamic type system, unary arithmetic instructions—bit manipulations—always update the type of `dst` register as `Scalar`. Double-operand instructions behave the same, except in specific cases. When adding or subtracting a `Scalar` to/from a `PtrType`, or adding a `PtrType` to `Scalar`, the destination register is set to `PtrType`.

Data handling operations. Data handling includes 64-bit `mov` and `loads`. `mov` instructions copy data between registers or from an immediate to a register, while `loads` place 64-bit constants (`PtrType` or `Scalar`) into registers. 64-bit `mov` and `load` update the destination register with the source type, while others change the destination register’s type to `Scalar`.

Memory operations. eBPF supports 1, 2, 4, and 8-byte general loads and stores with 4/8-byte atomic memory operations. Such operations typically require size-aligned memory

accesses. SPECHECK enforces strict type rules for memory operations to prevent pointer corruption, including partial pointer loads or overwrites.

Rather than tracking all memory slots, SPECHECK enforces two key constraints: accesses to general-purpose memory regions (*i.e.*, stack) must be size-aligned, and structured memory regions access must be size-aligned and contained within a single field. This approach leverages observations that the eBPF LLVM compiler performs stack spills at size-aligned offsets and structured memory fields are all size-aligned, allowing the checker to track pointers by examining only 8-byte aligned memory blocks. Additionally, atomic memory operations are restricted to concurrently accessed memory regions (*i.e.*, maps), while other memory regions, like the stack, being local to single program instances, do not require atomic operations.

Based on the above semantics and constraints, we now list the type rules that SPECHECK enforces for memory operations below:

- **load:** 8-byte memory loads from the memory slots with the same type α set the destination registers as α . Other loads set the destination registers to `Scalar`.
- **store:** Memory regions are modeled with either mutable types (*i.e.*, stack) or immutable types (*e.g.*, context). For stack, 8-byte stores replace the slot types with the source register's type. For stores smaller than 8 bytes: if the original 8-byte slot contained a pointer type, all 8 bytes are converted to `Scalar` to prevent partial pointer corruption. Otherwise, only the targeted bytes are marked as `Scalar`. For memory regions with immutable types, SPECHECK ensures slot types are compatible with the source register type—pointers can be stored as scalars, but not vice versa.
- **Atomic operations:** Maps—memory regions with the immutable type `Scalar`—are the only areas where atomic operations apply currently. Thus, their memory slot types remain unchanged and the register holding loaded data is always `Scalar` type.

Control flow operations. SPECHECK executes instructions sequentially, except for jumps, function calls, and exits. Jumps transfer control within a function, either unconditionally or based on comparison results. They generally do not change data types with one exception: comparisons (`==` and `!=`) between a `PtrOrNullType` and a `Scalar` with value 0, which changes the former to either `PtrType` (non-null) or `Scalar` with value 0 (`null`). Direct function calls pass up to five arguments via caller-saved registers `r1`–`r5`, while `r6`–`r9` are callee-saved. For kernel function calls, argument types are validated against the function type declarations upon entry, while other registers are set to `Uninit`. On function return, `r0` holds the return value, whose value range and type are constrained by program types to ensure correct interpretation at the attachment point. For instance, the 33 program

types (grouped into nine categories) each have defined return value constraints. After a function returns, callee-saved registers are restored, while caller-saved registers and callee stack slots are set to `Uninit`.

3.3 eBPF Safety Specification

SPECHECK derives safety properties through a top-down approach based on the CIA (Confidentiality, Integrity, and Availability) triad security model [35]. This systematic approach leads us to define three key aspects of safety:

- **Availability:** Unrestricted eBPF programs can compromise kernel availability in several ways: causing kernel crashes through memory errors, blocking kernel threads indefinitely, and depleting system resources through non-terminating execution or improper resource management. To protect kernel availability, we define three safety properties: control flow safety (SP1), memory safety (SP2), and resource safety (SP3).
- **Integrity:** Kernel integrity requires that eBPF programs do not modify unauthorized data—namely 1) kernel memory outside the eBPF VM and 2) eBPF registers or memory locations that are read-only or inaccessible. Formally, executing an instruction $insn$ on a state σ —satisfying the safety requirements for $insn$ —guarantees that unauthorized data (D_{ua}) remain unchanged between σ and the resulting state σ' .

$$\forall insn, \sigma, \sigma'. (safe(\sigma, insn) \wedge \sigma \xrightarrow{insn} \sigma') \Rightarrow \sigma \stackrel{D_{ua}}{\sim} \sigma'.$$

This integrity definition entails two safety properties: memory safety (SP2) and VM integrity (SP4).

- **Confidentiality:** eBPF programs running in the kernel context have access to sensitive kernel data, such as kernel pointers, information retrieved through kernel functions, and data in sensitive memory regions (*e.g.*, packets and contexts). These sensitive information must never be leaked to unprivileged users. Otherwise, attackers can exploit them to exfiltrate private data or escalate privileges with the help of other kernel vulnerabilities [7]. Therefore, the confidentiality (non-leakage) of the sensitive data becomes critical and we formally define it below.

$$\forall insn, \sigma_1, \sigma_2, \sigma'_1, \sigma'_2. (safe(\sigma_1, insn) \wedge safe(\sigma_2, insn) \wedge \sigma_1 \stackrel{U_\ell}{\sim} \sigma_2 \wedge \sigma_1 \xrightarrow{insn} \sigma'_1 \wedge \sigma_2 \xrightarrow{insn} \sigma'_2) \Rightarrow \sigma'_1 \stackrel{U_\ell}{\sim} \sigma'_2$$

Specifically, we partition the eBPF VM state into two disjoint domains: high-security U_h (sensitive data) and low-security U_ℓ (all other data). Only the low-security domain is observable to unprivileged users via public channels. Based on this, given any two states σ_1 and σ_2 that are safe to execute an instruction $insn$, if their data in the low-security domain U_ℓ is equal, the resulting states preserve the equivalence in the domain U_ℓ . Intuitively, this ensures

that no high-security information flows to low-security data, preventing leakage. This yields safety property SP5.

Overall, we define an eBPF program as safe if it satisfies the global control-flow safety and each of its instructions is safe regarding other safety properties SP2 to SP5.

We now detail the control-flow safety property and each per-instruction safety property separately.

SP1: Control flow safety. eBPF programs must terminate in finite time—in terms of the number of instructions executed—with an explicit `exit` instruction. Any execution of a program can have a maximum of 1M instructions for privileged users and 4096 instructions for unprivileged users. This ensures execution control returns properly to the kernel and prevents denial-of-service attacks, which could otherwise be launched through resource exhaustion.

SP2: Memory safety. SPECHECK models only discrete eBPF VM memory regions, so any unmodeled memory lies outside the VM. These unmodeled memory can be accessed either through out-of-bounds VM memory access or via kernel functions. In our specification, safety for unmodeled memory when accessed via kernel functions is out of scope. Such accesses pose the same risks as general kernel code, especially, some kernel functions are wrappers around native Linux functions. Thus, regulating eBPF behaviors in this context is unnecessary unless the entire Linux kernel is formally verified as memory-safe. Therefore, memory errors on unmodeled memory can only occur due to out-of-bounds eBPF VM memory access.

Based on the aforementioned memory modeling, we enforce three key memory safety constraints: First, any dereferenced pointer must be non-null, which is verified by checking if the pointer's type is `PtrType` before memory operations. Second, memory accesses must be within bounds and have valid permissions. Memory regions in eBPF VM either have fixed or dynamic bounds. The above conditions apply to regions with fixed bounds (e.g., stack) and dynamic regions (e.g., packet) where memory bounds are determined dynamically through pointer comparisons, such as between the packet and its end pointer `packet_end`. In contrast, other dynamic regions (e.g., buffer) depend on runtime checks. Third, to ensure temporal memory safety—preventing issues like use-after-free and double-free—we track the state of memory dynamically allocated through kernel functions. All related kernel functions are required to check the memory state before operations and update it afterward.

SP3: Resource safety. Resource safety ensures that all dynamically allocated resources are properly released before program termination. Specifically, before allowing program termination, SPECHECK checks that all dynamically allocated memory has been freed and all acquired resources (such as locks) have been released by examining the program's resource tracking state.

SP4: VM invariant/integrity. The eBPF VM enforces this invariant by making register `r10` read-only, prohibiting any instructions from writing to it as a destination register.

SP5: VM data safety. SPECHECK ensures data safety by adopting the capability model. In particular, kernel functions that retrieve sensitive information and sensitive memory regions are only allowed for specific program types, which require capabilities (e.g., `CAP_PERFMON`) to upload. Without these capabilities, SPECHECK enforces the following data safety properties to protect the sensitive data (i.e., pointers and uninitialized data).

First, it prohibits programs from reading data with the type `Uninit`, as it may contain private kernel data that was not erased. Second, it prohibits programs from storing pointers in public channels, such as map memory regions and helper calls that write to userspace memory. Still, eBPF programs can convert pointers (`PtrType` and `PtrOrNullType`) to scalars using arithmetic, memory, and call operations, potentially leaking these scalars to userspace. For instance, a bitwise or on a pointer converts it to a scalar according to the type rules described in §3.2. SPECHECK prevents such leaks by strengthening type rules that convert pointers to scalars.

- *Arithmetic instructions.* Arithmetic operations that take pointers but produce scalars are all disallowed.
- *Data handling operations.* Instructions that are not 64-bit `mov` and `load` and operate on pointers, are prohibited.
- *Memory operations.* The type rules in §3.2 allow converting pointers to scalars by loading or storing pointers smaller than 8 bytes. Such type transitions are prohibited to avoid kernel pointer leakage.
- *Control flow operations.* Jump instructions do not suffer from unsafe type conversions. Meanwhile, call instructions that pass arguments with pointer types to scalar parameters are already prevented in the defined type rules.

In addition, side channels can arise from two sources: (1) comparisons between pointers and scalars, and (2) speculative execution. To prevent information leakage through pointer-scalar comparisons, we prohibit such comparisons in both general jump instructions and atomic compare-and-exchange operations. But we allow `PtrOrNullType` to be compared with the scalar value 0 in equal/non-equal jumps to check if a pointer is null. For speculative execution side channels, we assume that the eBPF VM can rewrite programs with appropriate mitigations. Thus, we do not impose additional constraints on eBPF programs for this case.

Discussion of safety property completeness and soundness. The five safety properties outlined above are derived practically using a top-down approach, and the Linux eBPF verifier developer confirmed that they genuinely reflect the verifier's intended guarantees. We have verified that the encoded safety properties SP1-SP5 ensure the aforementioned

confidentiality and integrity definition. Any omission of these properties breaks the proofs. We are also actively working on a formal definition for availability, the last component of CIA, while empirical validation shows SpecCheck’s practical soundness regarding availability: (1) all corresponding upstream self-tests pass, (2) all historical bugs are caught, and (3) no false alarms after 3 months of fuzzing. *More importantly, these properties are sufficient to identify known bug types within our bug-finding scope.*

3.4 Specification Encoding

We adopt Dafny as our specification language, which has native support for encoding specifications, such as pre-conditions, post-conditions, pure functions/predicates, and lemmas. Moreover, the Dafny verifier can track violated constraints when these conditions fail, enhancing the debugging and refinement process.

Concretely, we define the eBPF VM state (e.g., memory and registers) as an immutable datatype. The specification of each instruction is implemented as a side-effect-free Dafny function, similar to a predicate but capable of returning values beyond just booleans. Per-instruction Dafny function takes an eBPF VM state as well as an instruction and returns a produced new state. Safety properties are encoded as functions’ pre-conditions, while function bodies express instruction semantics.

We encode each instruction’s specification based on its operational semantics from the ISA [9], the semantic constraints and type rules in §3.2, and safety properties in §3.3. For kernel functions whose semantics are not detailed in the ISA, we extract both operational semantics (e.g., return values) and semantic constraints (e.g., `spin_lock` requires no already held locks) from their kernel implementations. To prevent encoding errors in safe properties (e.g., missing safety properties), we map each safety property with its corresponding semantic component: 1) SP2 (memory safety) with memory read/write/allocation/release semantics, 2) SP3 (resource safety) with program termination semantics, 3) SP4 (VM integrity) with register write semantics, and 4) SP5 (data safety) for all instruction semantics. Using the mapping, we can identify the necessary safety properties of each instruction by matching its semantic with the safety properties’ semantic component.

Figure 5 illustrates the encoding of the 32-bit bitwise negation instruction `neg32`. Line 3 checks if the instruction is `neg32` as defined in our terms. The semantics of `neg32` involves register write, thus having the VM invariant **SP4** and the data safety property **SP5** as preconditions. SP4 requires the modified register (`dst`) must not be the read-only register `R10`. SP5 enforces this instruction to compute on scalar data if the privilege is missing, avoid leaking pointers or uninitialized values. If the safety properties hold, according to the type rule, the type of destination register transits to scalar. Further, the instruction semantic regarding data

```

1 function neg32(s: State, insn: Instruction) : State
2   // Pre-conditions
3   requires exists dst, src_imm ::
4     insn == ARITHUNARY(dst, NEG32)
5   requires insn.dst != R10 // SP4
6   requires get_reg_tv(s, insn.dst) != Uninit // SP5
7   requires !s.cfg.allow_ptr_leak ==> is_scalar(s, insn.dst)
8   {
9     // Instruction semantics
10    var dst_val := get_reg_arith_val(s, insn.dst);
11    var new_val := bvnot32(dst_val);
12    var new_tv := Scalar(Normal, new_val);
13
14    // Return a copy of state s with reg dst set to new_tv
15    new_state(s, insn.dst, new_tv)
16  }

```

Figure 5. The encoded specification of instruction `neg32`, which flips a 32-bit value in the register `dst`. Safety properties are specified as pre-conditions, whereas the instruction semantics are defined within the function body.

value is encoded with the negation operation on the input value (i.e., `bvnot32`). Specifically, SPECHECK does not directly model absolute addresses for values of type `PtrType` or `PtrOrNullType`. Instead, a pointer is represented as a triple (`r`, `memid`, `off`). We introduce an uninterpreted function—a function without a body—that maps each region to its unknown base address; thus, the absolute address of a pointer is its base plus the offset. Since the concrete memory layout is unknown, we constrain the base in the post-conditions of that uninterpreted function to keep the ranges of memory regions pairwise disjoint.

4 Marrying Specification with Fuzzing

With the encoded specification—per-instruction semantics and safety properties—defined in Dafny (§3), the next step is to turn SPECHECK into a bug oracle that can be integrated with fuzzers, which is the focus of this section. In particular, we show how to verify whether a concrete eBPF program is safe or not via shallow embedding [25] and present the complete fuzzing workflow in §4.2.

4.1 Shallow Embedding

Shallow embedding maps components of a source language S to corresponding elements of a target language D . This paper maps eBPF instructions to Dafny. Algorithm 1 outlines the embedding procedure. The algorithm performs a depth-first traversal of the program’s control flow graph, mapping each instruction to its corresponding method in the specification (line 5). During traversal, the algorithm handles control flow instructions specially: (1) For unconditional jumps, it jumps to the target instruction. For conditional jumps, it explores both the fallthrough path and the jump target path; (2) For pseudo function calls, it inlines them and jumps to the call entry and saves the location of the next instruction; (3) On the `exit` instruction, it restores any saved locations to explore remaining paths. The embedding procedure itself performs control flow validation, complementing the checks done by

Algorithm 1: The embedding process in SPECHECK

```

1 function Embed(insns, limit)
2   insnStack, idx, output ← [], 0, ""
3   while idx < limit do
4     insn = insn[idx]
5     output += ISA2Method(insn)
6     if UncondJump(insn) then
7       idx = JumpTarget(insn)
8     else if CondJump(insn) then
9       PushInsn(insnStack, JumpTarget(insn))
10      idx = idx + 1
11    else if PseudoCall(insn) then
12      ▶ Inline pseudo calls
13      PushInsn(insnStack, nextInsn)
14      idx = FunEntry(insn)
15    else if Exit(insn) then
16      if Empty(insnStack) then
17        break ▶ Break at the last instruction
18      else
19        idx = PopInsn(insnStack)
20    else
21      idx = idx + 1
22  return output

```

<pre> 1 // 2 r2 = *(u64 *) (r1 + 0) 3 r3 = 2 4 r2 %= r3 5 // 6 if r2 == 0 goto L1 7 ... 8 call map_lookup_elem 9 L1: 10 exit </pre>	<pre> 1 s0 := init_state(cfg); 2 s1 := load(s0, MEMLD(R2, R1, 0,)); 3 s2 := mov(s1, DATAMOVIMM(R3, 2,)); 4 s3 := mod(s2, ARITHBINREG(R2, R3,)); 5 s4 := jeq(s3, CONDJMPIMM(R2, 0,)); 6 if (!s4.jmp_res) { 7 ... 8 s5 := map_lookup_elem(s4'); 9 exit(s5); 10 } else { exit(s4); } </pre>
---	--

(a) eBPF assembly code. (b) Dafny code embedding eBPF.

Figure 6. The Dafny program shown in (b) is the embedding of the eBPF program in (a), which is in the form of eBPF assembly code. Each instruction in (a) is line-by-line mapped to (b). For brevity, we omit keywords (e.g., `var`) and partial instruction fragments.

the Dafny verifier. For loops, the algorithm uses bounded unrolling to ensure termination.

Figure 6 illustrates the shallow embedding approach, where eBPF assembly code is mapped line-by-line to the corresponding Dafny method invocations. For example, the instruction (line 2) loads a 64-bit value from the start of the context memory region (pointed to by register `r1`) into register `r2` and is embedded as a call to Dafny’s `load` function. Similarly, conditional jumps are embedded as `if` combined with appropriate comparison function calls in Dafny.

4.2 Overall Fuzzing Workflow

In this subsection, we present the integration of SPECHECK as a testing oracle within the VERITAS fuzzing framework.

Figure 7 shows the workflow of VERITAS. Initially, the syntax-based generator produces test cases following the eBPF ISA. Further, the executor runs test cases through

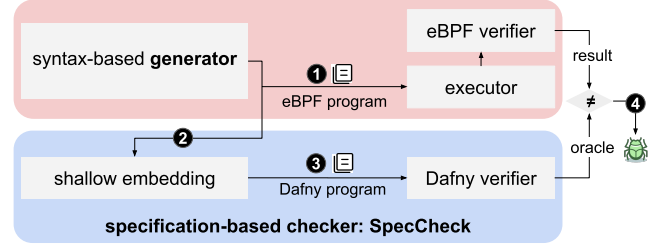


Figure 7. VERITAS overview. The generator produces syntax-valid eBPF programs as test cases. Further, the eBPF verifier takes them to verify their safety (1). Simultaneously, our checker (SPECHECK) embeds these test cases into Dafny programs based on our specification (2) and check their safety (3) through the Dafny verifier, which serves as an oracle. Finally, a difference between the eBPF verifier result and the SPECHECK oracle indicates a bug (4).

the eBPF verifier, which outputs the verification result, either accepting or rejecting programs (step 1). Simultaneously, VERITAS uses SPECHECK to independently validate the safety of the test cases against specifications encoded in Dafny. This involves embedding eBPF programs in Dafny (step 2) and running the Dafny verifier to ensure they comply with the specifications (step 3). Any discrepancy between the results of the eBPF verifier and SPECHECK indicates a bug in the eBPF verifier (step 4). Specifically, a discrepancy arises when (1) programs deemed safe by the eBPF verifier are flagged unsafe by SPECHECK, (2) the reverse occurs, or (3) both reject the program but cite different safety violations. In the third case, while the eBPF verifier’s final verdict on the program’s safety is correct, its subprograms may reveal incorrect verification results, belonging to one of the first two cases.

The Dafny verifier relies on SMT solvers to determine if the pre- and post-conditions hold. While SMT solvers are precise, they can be computationally expensive and may even timeout or become undecidable when faced with complex or numerous constraints [18]. We use three key insights to address the scalability challenge of SMT solvers.

Small test cases. Most eBPF verifier bugs can be triggered by small, simple programs rather than large, complex ones. Our evaluation in §6.1 shows that the proof-of-concept (PoC) programs for existing eBPF verifier bugs are relatively small, mostly containing only 1–30 instructions. This insight allows us to configure our test case generator to focus on generating small programs, which reduces SMT-solving time while maintaining effectiveness at finding bugs.

Incremental state sampling for fast verification. When the eBPF verifier rejects a program, it identifies the unsafe / culprit instruction. To check if the rejection is correct, we only need to verify the safety of that instruction rather than checking the entire program. However, SPECHECK still needs information about the VM state immediately before that instruction executes. We instrument the eBPF verifier

to generate VM states incrementally at configurable intervals during its verification process. We then use the sampled state closest to the culprit instruction as the initial state for SPECHECK. This sampling approach significantly improves performance, as we demonstrate in §6.3.

Parallel verification for high throughput. To further improve the fuzzing throughput on top of the above two solutions, VERITAS runs the checker asynchronously from the fuzzing loop and simultaneously on multiple test cases. Although the insights help, 0.2% of tests still time out (see §6.1). However, it is not a concern in fuzzing. If a test that might trigger a bug times out, we simply skip it, as another solvable test will likely reveal the same bug.

5 Implementation

We integrate VERITAS into Syzkaller [23] to leverage its existing fuzzing components (e.g., executor). The implementation of each component in VERITAS is detailed below.

SPECHECK. The specification is encoded in Dafny [10] with 2000 lines of code—roughly nine engineer weeks—and the embedding procedure is implemented in 600 lines of C++. SPECHECK models all 171 op-codes recorded in the eBPF ISA RFC [9], except for *deprecated* instructions (e.g., BPF_IND), which were introduced specifically to access packet data in classic BPF. The remaining gap is in kernel helpers, where we deliberately focus on the 50 most frequently used of 455 functions ($\approx 11\%$). As writing kernel function specifications becomes increasingly labor-intensive and the eBPF ecosystem expands, we are exploring scalable automation techniques to efficiently generate the specification for kernel functions with minimal manual effort.

eBPF program generator. The generator, written in 2,100 lines of C++ code, produces a random control graph with vertices as basic blocks, iterates over each to generate syntax-valid instructions adhering to basic safety rules (e.g., avoiding r10 as a destination register), and records the program state coarsely, including initialized registers and their types.

eBPF verifier state sampling. We patch the Linux kernel to sample verifier state every N instructions, capturing registers, stack slots, and spin_lock statuses, without altering verifier logic. The state is shared to userspace via debugfs.

6 Evaluation

We evaluate VERITAS on the Linux eBPF verifier to answer the following research questions.

- Q1. How effective and accurate is VERITAS in finding eBPF verifier bugs? (§6.1)
- Q2. How does SPECHECK outperform other oracles? (§6.2)
- Q3. What is the testing performance of VERITAS? (§6.3)
- Q4. What is the required effort to extend the specification for new instructions? (§6.4)

Experiment setup. We evaluate VERITAS on Ubuntu 22.04.4 LTS with a 224-core Intel(R) Xeon(R) Platinum 8276L processor and 754G memory. Each fuzzer instance runs in isolation on the same 224-core server with 754 GB RAM to eliminate resource bias. We use Dafny CLI V4.6.0 and Z3 V4.12.1 [8] to verify embedded eBPF programs in Dafny.

6.1 Bug-Hunting Result

After running VERITAS intermittently for three months, it uncovered 15 bugs: three cases where unsafe eBPF programs are accepted, nine cases where safe eBPF programs are rejected, one case where programs misusing atomic instructions on local memory are accepted (bug #9, see analysis in §6.5), listed in Table 1, as well as one memory bug and one undefined behavior identified through KASAN and UBSAN in the verifier itself, though the latter two are beyond the focus of the work. These bugs, found despite extensive testing by maintainers and prior research, underscore the incapability of existing oracles. We reported all 15 bugs, of which 12 are acknowledged. The remaining three (bugs #10, #11, and #13) are usability issues rooted in the core design limitations of the verifier. These are typically considered low priority by developers compared to security issues and eBPF ISA extensions and are therefore often overlooked. We emphasize that VERITAS not only identifies misimplementation bugs, but also highlights existing design limitations, offering insights that improve the verifier in the future. Eight bugs were fixed quickly. The rest are pending resolution for two reasons: (1) some require non-trivial code refactoring or new algorithms, which demands significant development time, and (2) others represent usability issues rather than security vulnerabilities, which maintainers have prioritized lower in their roadmap. We are actively collaborating with eBPF maintainers to develop and submit patches for the outstanding issues.

Further, we summarize the characteristics of bugs below:

Critical consequences. New bugs can lead to severe security attacks and usability issues. For example, bug #1 and #2 can be exploited by users only with CAP_BPF to escalate into root and leak kernel pointers, breaking KASLR. Moreover, bug #6 took a sched-ext developer hours to debug it.

Diverse culprit instructions. New bugs lie in the incorrect verification of all instruction categories, e.g., arithmetic (bug #7), data handling (bug #1), memory (bug #5), and control flow operations (bug #2), demonstrating SPECHECK is general enough for finding bugs throughout the entire verifier.

Small-sized test cases. Test cases revealing new bugs are as small as two instructions, as demonstrated by bugs #3, #5, and #7. To validate that eBPF verifier bugs can be detected with small test cases, we analyzed the size of various test cases, including eBPF self-tests and collected bug proofs-of-concept (PoCs). As shown in Figure 8, most test cases contain 1-30 instructions (a) and 1-9 execution paths (b), with only 23 and 28 test cases exceeding 30 instructions and ten execution

RC	#	Instruction	Status	Description and Consequence
RC4	1	mov	Fixed	Fails to track non-r10 precision on stack, leading to privilege escalation.
RC3	2	kfunc call	Fixed	Miss argument type checks, leading to DoS.
RC4	3	store	Fixed	Incorrectly mark stack slot type, leading to ASLR bypass.
RC3	4	atomic*	Fixed	Miss propagating precisions to stack slots used in atomic instructions.
RC3	5	atomic_xchg	Acked	Verifier misidentifies scalar type, failing stack pointer validation.
RC1	6	store	Acked	Not propagate scalar range from registers to stack
RC3	7	be32	Fixed	Incorrect precision back-propagation
RC3	8	store	Fixed	Mis-reject a 32-bit store to overwrite a spilled 64-bit scalar on the stack.
RC2	9	atomic*	Acked	Allow atomic instructions operating on local memory regions.
RC2	10	arith operations	Reported	Inconsistent constraints on instructions converting pointer to scalars
RC1	11	jumps	Reported	Coarse-grained pointer comparison
RC1	12	memory operations	Acked	Imprecise stack data tracking
RC1	13	arith operations	Reported	Inaccurate tracking of arithmetic instruction result

Table 1. The list of bugs detected by VERITAS. kfunc call: kernel function call

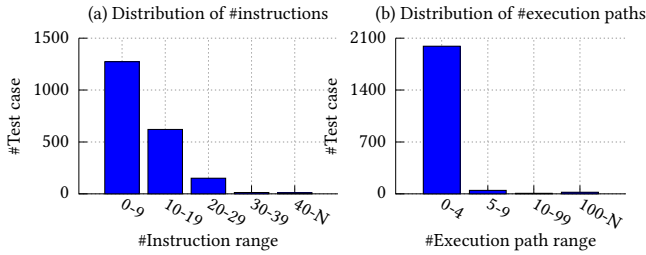


Figure 8. The size of test cases (self-tests and bug PoCs) measured by instruction count and execution paths, indicating that small-sized test cases are sufficient for finding bugs.

paths, respectively. These findings support the insight in §4 that small test cases are sufficient to uncover most bugs.

Accuracy of the specification-based oracle. During the entire intermittent running, SPECHECK reported no false alarms but encountered timeouts (0.2% of tests) due to test complexity. These timeouts occur when the Dafny verifier encounters undecidable constraints or exceeds 900 seconds.

6.2 Comparison with the State-of-the-art

To measure SPECHECK, we compare it with the oracle used in existing fuzzers [26, 27, 36, 37] from two perspectives.

First, we evaluate if VERITAS can detect bugs identified by existing fuzzers. This paper focuses on the oracle SPECHECK—instead of a test generator—which can be integrated into other fuzzers to leverage their test case generator with engineering effort. Therefore, we directly evaluate SPECHECK using PoCs from a collected bug dataset. The dataset includes 14 verifier bugs reported by existing fuzzers: 10 from SEV, 1 from BVF, 2 from BUZZER, and 1 from BRF. We excluded bugs outside VERITAS’s scope, such as those in the JIT compiler or kernel function implementations. While SEV claimed 15 bugs (comprising 12 incorrectly accepted unsafe programs and 3 incorrectly rejected safe programs), we could only identify 11 of these through mailing lists, as individual bugs were poorly documented and difficult to trace. It’s worth noting that SEV’s oracle has an inherent limitation:

it cannot detect incorrectly rejected safe programs as its oracle relies on the runtime states but rejected programs even have no chance to run. SEV identified incorrectly rejected safe programs either manually or through the verifier’s self-assertions. We also excluded two additional “bugs” from SEV and BVF that only produced misleading error messages without affecting verification outcomes. SPECHECK detected all 14 bugs in the dataset, demonstrating its comprehensive capability in identifying existing verifier bugs.

We also evaluated if existing fuzzers can detect the bugs found by VERITAS. Among the four fuzzers considered, only BRF [27] and BUZZER [26] are open-source, allowing for direct empirical comparison, while SEV [36] and BVF [37] were evaluated through theoretical analysis. To ensure a fair comparison that eliminates randomness in test generation, we provided the exact bug PoCs directly to BUZZER and BRF. Despite this advantage, neither fuzzer successfully detected any of the bugs found by VERITAS. Even for bug #1, which can potentially cause runtime memory errors, both fuzzers failed to find the correct map data to trigger the faulty instruction in the provided PoC. Our theoretical analysis on SEV and BVF indicates that existing fuzzers may detect bug #1, linked to runtime memory errors. The remaining bugs do not manifest as runtime errors, which clearly demonstrates the necessity of our specification-based oracle. These results conclusively show that SPECHECK can identify bugs that are beyond the detection capabilities of existing oracles.

6.3 Fuzzing Performance

We ran the fuzzer for 40 hours, generating eBPF programs with 5-30 instructions based on test case size analysis in §6.1, with a Dafny verification timeout of 900 seconds and state sampling every three instructions after six instructions.

Fuzzing throughput. State sampling and asynchronous checking during evaluation achieve a fuzzing speed of 23-25 tests per second by utilizing all 224 cores. Compared with BUZZER (700 tests/s) and BRF (50 tests/s), VERITAS trades

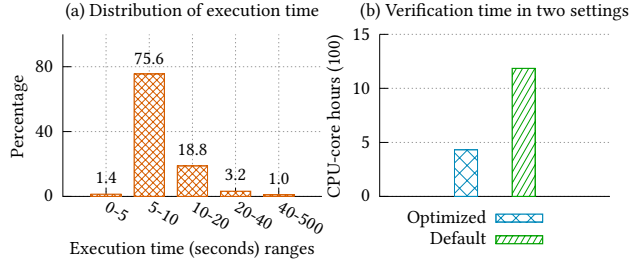


Figure 9. Fuzzing performance. (a) shows the entire execution time distribution of test cases during fuzzing, while (b) depicts the Dafny verification time without and with sampled verifier states.

throughput for a much higher bug-per-test rate: 12/13 newly discovered bugs are silent semantic errors rather than the crash-triggering faults that raw throughput alone can expose.

The checking time of each test case in SPECHECK, including state sampling, embedding, and Dafny verification, takes 1.5 to 489 seconds, averaging 10 seconds. As shown in Figure 9 (a), 95.8% of test cases complete within 5-20 seconds, with few exceeding 20 seconds. Stage-wise analysis reveals Dafny verification consumes 99% of the time, while state sampling and embedding account for just 0.03% and 0.0002%.

Performance improvement from state sampling. We reran the fuzzer for 40 hours with the same setting in the above evaluation, except verifying each test case twice: once with state sampling (*optimized setting*) and once by verifying the complete test case without state sampling (*default setting*). VERITAS generated 143,733 test cases, with 51,254 using state sampling. On average, state sampling saved 53 seconds per test case, with a maximum saving of 12 minutes. However, state sampling introduces longer checking time on 5.0% (2,574) test cases because SPECHECK identifies less complex violated constraints in another culprit instruction in the default setting. Despite this, state sampling reduces overall resource usage, saving 754 CPU-core hours¹, as illustrated in Figure 9 (b). Moreover, state sampling does not introduce bug misses in the evaluation.

Code coverage. VERITAS aims to provide a comprehensive testing oracle rather than a dedicated test generator like BVF and BRF, with test generation improvements planned for future work. Nevertheless, our system achieves 32% branch coverage (3,432 branches), which exceeds BRF’s 29% (3,124 branches)—BRF being the only publicly available system with comparable metrics. This coverage level has proven sufficient for discovering numerous bugs. It is worth noting that our coverage measurement uses XOR operations on basic block IDs, a method that may introduce hash collisions and potentially underestimate the actual coverage achieved.

6.4 Specification Extensibility and Complexity

To extend the specification with new instructions or kernel functions, users are expected to write more ghost methods

¹A CPU-core hour measures the usage of one CPU core for an hour [14].

in Dafny, embedding safety rules as pre-conditions and operational semantics as post-conditions. While it is hard to measure the actual manual effort scientifically, we estimate it by counting the pre- and post-conditions required for each instruction in the current specification.

On average, each instruction or kernel function has 3.6 pre-conditions (max 13) and 3.7 post-conditions (max 6). Arithmetic and data-handling instructions have fewer pre-conditions (2.9) but more post-conditions (4.7), because they generally only require operand type checks but involve special pointer semantics due to representing pointers as memory regions and offsets. Conversely, memory operations and kernel functions generally have more pre-conditions (4.7) and fewer post-conditions (2.3). The more pre-conditions stem from fine-grained memory access control and program state checks, while fewer post-conditions arise because general-purpose memory regions, excluding stack memory, have simpler type rules. Additionally, kernel functions typically only specify return values as post-conditions, except in complex cases, such as “spin_lock”, which often involve changes to program states (e.g., lock state).

6.5 Case Study

We present three additional bugs found by VERITAS to further illustrate their characteristics.

```

1      r1 = r10
2      if cond
3          (b1) / \ (b2)
4          /      \
5      *(u64 *) (r1 - 120) = 0  *(u64 *) (r1 - 120) = r2
6          \      /
7      r2 = *(u64 *) (r1 - 120)
8      // r3 points to an eight-byte memory region
9      r3 += r2
10     *(u64 *) (r3 + 0) = evil_data
11 // Memory write from (b2) can overwrite any kernel addresses.

```

Figure 10. An erroneous optimization in Linux eBPF verifier prunes instructions after line 7 in branch b2, causing out-of-bound access.

Bug #1: Out-of-bound access leading to privilege escalation. The bug in Figure 10 is a mis-optimization bug originating from RC4. It overlooks r2 and stack slot r1-120 as critical states, skips their comparison at the convergence point (line 7) when verifying branch (b2), and incorrectly deems the program states from branches (b1) and (b2) equal, leading to skipped verification of instructions beyond line 7 in branch (b2). This allows branch (b2) to access any kernel address via r2, enabling privilege escalation with only the CAP_BPF capability. The vulnerability has been fixed and merged to the stable Linux version.

```

1  uint32_t array[4];
2  array[0] = 1; // fp-8
3  array[1] = 2; // fp-4
4  // Reject: attempt to corrupt spilled pointer on stack

```

Figure 11. Initialization of a 32-bit integer array is mistakenly rejected by the Linux eBPF verifier. fp represents the stack frame pointer. fp-8 means the stack slot at fp-8.

Bug #8: Mis-rejecting 32-bit variable initialization. In Figure 11, the eBPF verifier incorrectly rejects the initialization of a 32-bit integer array, citing *"attempt to corrupt spilled pointer on stack"*, if the program has no data leakage privilege. This error arises because the verifier assumes an 8-byte aligned stack slot contains a pointer without verifying its type. Specifically, after initializing `array[0]` at `fp-8` with the integer constant 1, the slot type is a scalar, not a pointer. When initializing `array[1]` at `fp-4`, the verifier checks the 8-byte region (`[fp-8, fp-1]`) for pointers to prevent partial overwrites and potential pointer leakage. However, it does not check the type of the region and erroneously treats it as a pointer, leading to the rejection. Without SPECHECK, no existing oracles can detect such bugs.

Bug #5 and #9. Atomic operations are intended for concurrent access to shared memory. However, the verifier allows unnecessary atomic instructions on private memory (e.g., stack), as in the acknowledged bug #9. Although not directly causing usability or security issues, this increases implementation complexity and contributes to bug #5. The issue arises with `atomic_xchg(r1, r1)` in privileged mode, where `r1` is a stack pointer. This instruction swaps the value at the address pointed to by `r1` with the value in `r1`, involving a load followed by a store. The verifier checks the load and then the store, but immediately marks `r1` as a scalar after the load, leading to a failure in the store check since `r1` is no longer a pointer. Finally, this safe instruction is mistakenly rejected.

7 Discussion

Semantics fidelity. To ensure that SPECHECK is faithfully modeling eBPF, we tested that: 1) SPECHECK conforms to all selftests of the Linux eBPF verifier, and 2) all discrepancies between SPECHECK and the Linux eBPF verifier found during fuzzing can be attributed.

Future Maintenance. We aim to strengthen the specification’s soundness and completeness by formalizing additional higher-level properties. Moreover, we plan to integrate the specification into the eBPF development CI/CD pipeline, enabling synchronized updates with the eBPF verifier modifications and automated validation against previously verified higher-level properties to preserve correctness guarantees. To minimize ongoing effort, we can leverage large language models (LLMs) to draft the specifications of kernel functions—components under active development—which developers then review and finalize.

Beyond testing oracle. Beyond acting as a testing oracle, the specification in SPECHECK can be further leveraged to enable the automatic generation of formal proofs in proof-carrying code (PCC) [31, 32] in eBPF-based kernel extensions. PCC requires code producers (e.g., extension developers) to provide formal proof of code safety, allowing consumers (e.g., OS kernels) to verify compliance with safety properties, offering developers more flexibility in proving code safety.

Handling loops in SPECHECK. SPECHECK verifies eBPF programs with loops using bounded loop unrolling, which is sufficient as a testing oracle as most of the generated eBPF programs are small as shown in the bugs found in §6.1. An alternative is automated loop invariant inference or generating eBPF programs based on pre-defined loop invariants. We leave them to future work.

8 Related Work

We discussed related work on eBPF verifier testing in §2.3 and address the remaining related work below.

eBPF verification. Formal verification has improved the correctness of the eBPF JIT compiler [33] and range-tracking in the verifier [39]. Verifying the eBPF verifier is more complex due to its large codebase (e.g., 20,000 lines of code) and intricate logic, compared to JIT’s 2,440 lines for one architecture [5]. While we do not verify the verifier, our specification, defining instruction semantics and safety properties, can be leveraged to verify eBPF verifier further.

Specification-based testing oracle. The challenge of specification-based oracles involves defining the specification and encoding it as an executable oracle. Some studies offer frameworks [16, 20] and languages [1] that support these stages with assertions, while others focus on encoding using informal specifications such as POSIX specification or TCP/IP RFCs—either generating formal specifications automatically (e.g., rule-based methods [30]) or manually encoding them, as seen in Netsem [17], SibyIFS [34], Hydra [28], and Monarch [29]. In the context of eBPF, the lack of formal documentation forces us to create a systematic specification, a difficult yet key contribution. Moreover, our per-instruction specification with safety properties as pre-conditions and instruction semantics as function bodies introduces an extensible approach that integrates seamlessly with shallow embedding for fuzzing-based testing oracles.

9 Conclusion

Specification-based oracle offers a holistic approach to detecting correctness bugs. In the Linux eBPF verifier case, VERITAS finds high-impact bugs across diverse root causes, some of which can never be found with bug oracles in prior fuzzing campaigns as these oracles only indirectly hint the existence of a bug without revealing its root cause.

10 Acknowledgments

We thank our shepherd Chang Lou and the anonymous reviewers for their insightful feedback. We also thank Clément Pit-Claudel, Jiacheng Ma, Sankalp Gambhir, and Yawen Guan for their helpful feedback at various stages. This work was supported in part by the eBPF Foundation, the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (grant agreement No. 850868), NSERC (RGPIN-2022-03325), and NCC (2024-1488).

References

- [1] Test Oracles. <http://ix.cs.uoregon.edu/~michal/pubs/oracles.pdf>, 2001.
- [2] Kernel address space layout randomization. <https://lwn.net/Articles/569635/>, 2013.
- [3] Avoid verifier failure for 32bit pointer arithmetic. <https://lore.kernel.org/bpf/20200618234631.3321118-1-yhs@fb.com/>, 2020.
- [4] A discussion mail in the eBPF mailing list. <https://lore.kernel.org/bpf/20230502005218.3627530-1-drosen@google.com/>, 2023.
- [5] Complexity of the BPF Verifier. <https://pchaiguo.github.io/ebpf/2019/07/02/bpf-verifier-complexity.html>, 2023.
- [6] CVE-2023-2163 of the eBPF verifier that leads to privilege escalation. <https://nvd.nist.gov/vuln/detail/cve-2023-2163>, 2023.
- [7] Leak kernel pointers by exploiting spectre in eBPF programs. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf.git/commit/?id=e4f4db47794c>, 2023.
- [8] The Z3 Theorem Prover . <https://github.com/Z3Prover/z3>, 2024.
- [9] BPF Instruction Set Architecture (ISA). <https://www.rfc-editor.org/rfc/rfc9669.html>, 2024.
- [10] Dafny Reference Manual. <https://dafny.org/dafny/DafnyRef/DafnyRef>, 2024.
- [11] eBPF verifier source code. <https://github.com/torvalds/linux/blob/master/kernel/bpf/verifier.c>, 2024.
- [12] Sched_ext Schedulers and Tools. <https://github.com/sched-ext/scx>, 2024.
- [13] The eBPF selftest set. <https://github.com/torvalds/linux/tree/master/tools/testing/selftests/bpf>, 2024.
- [14] What are Core-Hours? How are they estimated? <https://support.onscale.com/hc/en-us/articles/360013402431-What-are-Core-Hours-How-are-they-estimated>, 2024.
- [15] Linux eBPF. <https://ebpf.io/>, 2025.
- [16] Shadi G Alawneh and Dennis K Peters. Specification-based test oracles with junit. In *CCECE 2010*, pages 1–7. IEEE, 2010.
- [17] Steve Bishop, Matthew Fairbairn, Hannes Mehnert, Michael Norrish, Tom Ridge, Peter Sewell, Michael Smith, and Keith Wansbrough. Engineering with logic: Rigorous test-oracle specification and validation for tcp/ip and the sockets api. *Journal of the ACM (JACM)*, 66(1):1–77, 2018.
- [18] Cristina Borralleras, Daniel Larraz, Enric Rodríguez-Carbonell, Albert Oliveras, and Albert Rubio. Incomplete smt techniques for solving non-linear formulas over the integers. *ACM Transactions on Computational Logic (TOCL)*, 20(4):1–36, 2019.
- [19] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.
- [20] David Coppit and Jennifer M Haddox-Schatz. On the use of specification-based assertions as test oracles. In *29th Annual IEEE/NASA Software Engineering Workshop*, pages 305–314. IEEE, 2005.
- [21] Jonathan Corbet. BPF at Facebook (and beyond). <https://lwn.net/Articles/801871/>, 2019.
- [22] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 238–252, 1977.
- [23] David Drysdale. Coverage-guided kernel fuzzing with syzkaller, 2016.
- [24] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. Simple and precise static analysis of untrusted linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 1069–1084, 2019.
- [25] Jeremy Gibbons and Nicolas Wu. Folding domain-specific languages: deep and shallow embeddings (functional pearl). In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*, pages 339–347, 2014.
- [26] Google. Buzzer - an ebpf fuzzer toolchain. <https://github.com/google/buzzer>.
- [27] Hsin-Wei Hung and Ardalan Amiri Sani. Brf: Fuzzing the ebpf runtime. *Proceedings of the ACM on Software Engineering*, 1(FSE):1152–1171, 2024.
- [28] Seulbae Kim, Meng Xu, Sanidhya Kashyap, Jungyeon Yoon, Wen Xu, and Taesoo Kim. Finding Semantic Bugs in File Systems with an Extensible Fuzzing Framework. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, Ontario, Canada, October 2019.
- [29] Tao Lyu, Liyi Zhang, Zhiyao Feng, Yueyang Pan, Yujie Ren, Meng Xu, Mathias Payer, and Sanidhya Kashyap. Monarch: A fuzzing framework for distributed file systems. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 529–543, 2024.
- [30] Manish Motwani and Yuriy Brun. Automatically generating precise oracles from structured natural language specifications. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 188–199. IEEE, 2019.
- [31] George C Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 106–119, 1997.
- [32] George C Necula and Peter Lee. Safe kernel extensions without runtime checking. In *OSDI*, volume 96, pages 229–243, 1996.
- [33] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. Specification and verification in the field: Applying formal methods to bpf just-in-time compilers in the linux kernel. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 41–61, 2020.
- [34] Tom Ridge, David Sheets, Thomas Tuerk, Andrea Giugliano, Anil Madhavapeddy, and Peter Sewell. Sibylfs: formal specification and oracle-based testing for posix and real-world file systems. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*, Monterey, CA, October 2015.
- [35] Spyridon Samonas and David Coss. The cia strikes back: Redefining confidentiality, integrity and availability in security. *Journal of Information System Security*, 10(3), 2014.
- [36] Hao Sun and Zhendong Su. Validating the ebpf verifier via state embedding. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 615–628, 2024.
- [37] Hao Sun, Yiru Xu, Jianzhong Liu, Yuheng Shen, Nan Guan, and Yu Jiang. Finding correctness bugs in ebpf verifier with structured and sanitized program. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 689–703, 2024.
- [38] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. Sound, precise, and fast abstract interpretation with tristate numbers. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 254–265. IEEE, 2022.
- [39] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. Verifying the verifier: ebpf range analysis verification. In *International Conference on Computer Aided Verification*, pages 226–251. Springer, 2023.
- [40] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. Fuzzing: a survey for roadmap. *ACM Computing Surveys (CSUR)*, 2022.

A Appendix

A.1 The abstracted syntax of eBPF instruction in SPECHECK

```

datatype Instruction =
  | ARITHUNARY(dst: REG, uop: ARITHUNARYOP)
  | ARITHBINREG(dst: REG, src: REG, binop: ARITHBINOP)
  | ARITHBINIMM(dst: REG, src_imm: bv64, binop: ARITHBINOP)
  | DATAMOVIMM(dst: REG, src_imm: bv64, moviop: MOVIMMOP)
  | DATAMOVREG(dst: REG, src: REG, movrop: MOVREGOP)
  | MEMLD(dst: REG, src: REG, ioff: s16, size: SIZE, sign_ext: bool)
  | MEMSTX(dst: REG, src: REG, ioff: s16, size: SIZE)
  | MEMST(dst: REG, src_imm: bv64, ioff: s16, size: SIZE)
  | ATOMICLS(dst: REG, src: REG, ioff: s16, asize: ASIZE, op: ATOMICOP)
  | CONDJMPREG(dst: REG, src: REG, jmpop: JMPPOP)
  | CONDJMPIMM(dst: REG, src_imm: bv64, jmpop: JMPPOP)
  | CALL_PSEUDO_FUNC() | CALL_HELPER() | CALL_KFUNCS() | EXIT()

datatype REG = | R0 | R1 | R2 | R3 | R4 | R5 | R6 | R7 | R8 | R9 | R10
datatype SIZE = | B | HW | W | DW
datatype ASIZE = | W | DW
datatype ARITHUNARYOP =
  | NEG32 | BV2BE16 | BV2BE32 | BV2LE16 | BV2LE32 | BV2SWAP16 | BV2SWAP32 | NEG64 | BV2BE64 | BV2LE64 | BV2SWAP64
datatype ARITHBINOP =
  | ADD32 | SUB32 | MUL32 | DIV32 | SDIV32 | MOD32 | SMOD32 | BVOR32 | BVAND32 | BVXOR32 | BVLSHR32 | BVASHR32 | BVSHL32
  | ADD64 | SUB64 | MUL64 | DIV64 | SDIV64 | MOD64 | SMOD64 | BVOR64 | BVAND64 | BVXOR64 | BVLSHR64 | BVASHR64 | BVSHL64
datatype MOVIMMOP =
  | MOVIMM32 | MOVIMM64 | LOADIMM64 | LOADMAPBYFD | LOADMAPVALBYFD | LOADMAPBYIDX
  | LOADMAPVALBYIDX | LOADVARBYBTFFID | LOADFUNCBYIDX
datatype MOVREGOP =
  | MOV32 | MOVSX8TO32 | MOVSX16TO32 | MOV64 | MOVSX8TO64 | MOVSX16TO64 | MOVSX32TO64
datatype ATOMICOP =
  | ATOMIC_ADD | ATOMIC_AND | ATOMIC_OR | ATOMIC_XOR | ATOMIC_FETCH_ADD | ATOMIC_FETCH_AND
  | ATOMIC_FETCH_OR | ATOMIC_FETCH_XOR | ATOMIC_XCHG | ATOMIC_CMPXCHG
datatype JMPPOP =
  | JEQ32 | JNE32 | JSET32 | JGT32 | JGE32 | JSGT32 | JSGE32 | JLT32 | JLE32 | JSLT32 | JSLE32
  | JEQ64 | JNE64 | JSET64 | JGT64 | JGE64 | JSGT64 | JSGE64 | JLT64 | JLE64 | JSLT64 | JSLE64

```

Figure 12. The abstracted syntax of eBPF instruction used in SPECHECK. The abstracted terms exclude the direct-jump instructions JA and JA32 and calls to pseudo functions: the former are translated to Dafny’s control-flow constructs, and the latter are inlined during shallow embedding.

A.2 The eBPF program state in SPECHECK

```

datatype State = State (
  R0: ETYPEV, R1: ETYPEV, R2: ETYPEV, R3: ETYPEV, R4: ETYPEV, R5: ETYPEV,
  R6: ETYPEV, R7: ETYPEV, R8: ETYPEV, R9: ETYPEV, R10: ETYPEV,
  R6': ETYPEV, R7': ETYPEV, R8': ETYPEV, R9': ETYPEV,
  cfg: ConfigState, jmp_res: bool, maps_meta: seq<MapState>, mems: seq<seq<MemSlot>>,
  ...
)
datatype MemSlot = MemSlot(field_perm: ACCESSPERM, etypev: ETYPEV, field_size: int)
datatype Mem = Mem(mem_type: MEMTYPE, is_concur: bool, base: bv64, data: seq<MemSlot>)

```

Figure 13. The eBPF program state in SPECHECK. It includes the register, memory, and branch informations. For brevity, we omit the state definition of eBPF VM configuration: ConfigState.

A.3 Typing rules of the dynamic type system

We present the way dynamic types are propagated by each instruction through dynamic typing rules in [Figure 14](#) in the form $\frac{\text{preconditions}}{S \rightarrow S[X.T \mapsto T']}$, which indicates that the eBPF VM state transitions from S to an updated state *only when the specified type preconditions are satisfied*. We only specify the way types flow and omit what happens to the values, since values follow standard eBPF semantics.

To interpret these rules, we highlight the below points:

- The left-hand sides are terms, whereas the right-hand sides are type rules. Term variables range over all possible values unless otherwise constrained. For example, uop ranges over the unary arithmetic operations defined in `ARITHUNARYOP`.
- Each rule must adhere to the semantic constraints defined in [§3.2](#), such as enforced memory alignment.
- If, during the execution of an instruction in an eBPF program, no rule matches in a top-to-bottom pass over the rule list, `SPECHECK` would dynamically report an error for this instruction.
- The basic type rules described in [§3.2](#) correspond to the rules without boxed preconditions. To ensure data safety, as discussed in [§3.3](#), we strengthen these rules by adding type preconditions shown inside boxes $\boxed{}$.

ARITHUNARY(dst, uop)	$\frac{S.R_{dst} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]}$
ARITHBINREG(dst, src, ADD64)	$\left\{ \begin{array}{l} \frac{S.R_{dst} = \text{Scalar}(_, _) \wedge S.R_{src} = \text{PtrType}(_, _)}{S \rightarrow S[R_{dst} \mapsto \text{PtrType}(_, _)]} \\ \frac{S.R_{dst} = \text{PtrType}(_, _) \wedge S.R_{src} = \text{Scalar}(_, _)}{S \rightarrow S[R_{dst} \mapsto \text{PtrType}(_, _)]} \\ \frac{S.R_{dst} \neq \text{Uninit} \wedge S.R_{src} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _) \wedge S.R_{src} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]} \end{array} \right.$
ARITHBINIMM(dst, src_imm, ADD64)	$\left\{ \begin{array}{l} \frac{S.R_{dst} = \text{PtrType}(_, _) \vee S.R_{dst} = \text{Scalar}(_, _)}{S \rightarrow S} \\ \frac{S.R_{dst} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]} \end{array} \right.$
ARITHBINREG(dst, src, SUB64)	$\left\{ \begin{array}{l} \frac{S.R_{dst} = \text{PtrType}(_, _) \wedge S.R_{src} = \text{Scalar}(_, _)}{S \rightarrow S} \\ S.R_{dst} \neq \text{Uninit} \wedge S.R_{src} \neq \text{Uninit} \wedge \\ \left(\boxed{(S.R_{dst} = \text{Scalar}(_, _) \wedge S.R_{src} = \text{Scalar}(_, _))} \vee \right. \\ \left. \boxed{(S.R_{dst} = \text{PtrType}(_, _) \wedge S.R_{src} = \text{PtrType}(_, _))} \right) \\ \hline S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)] \end{array} \right.$
ARITHBINIMM(dst, src_imm, SUB64)	$\left\{ \begin{array}{l} \frac{S.R_{dst} = \text{PtrType}(_, _) \vee S.R_{dst} = \text{Scalar}(_, _)}{S \rightarrow S} \\ \frac{S.R_{dst} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]} \end{array} \right.$
ARITHBINREG(dst, src, binop), binop $\notin \{\text{ADD64}, \text{SUB64}\}$	$\frac{S.R_{dst} \neq \text{Uninit} \wedge S.R_{src} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _) \wedge S.R_{src} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]}$
ARITHBINIMM(dst, src_imm, binop), binop $\notin \{\text{ADD64}, \text{SUB64}\}$	$\frac{S.R_{dst} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]}$

(a) Type rules of arithmetic operations.

DATAMOVREG(dst, src, MOV64)	$\frac{S.R_{src} \neq \text{Uninit}}{S \rightarrow S[R_{dst} \mapsto R_{src}]}$
DATAMOVREG(dst, src, movrop), movrop $\neq$ MOV64	$\frac{S.R_{src} \neq \text{Uninit} \wedge \boxed{S.R_{src} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)]}$
DATAMOVIMM(dst, src_imm, moviop)	$\frac{\begin{cases} S \rightarrow S[R_{dst} \mapsto \text{PtrType}(_, _, _)] \text{ if } \text{moviop} \in \{\text{LOADMAPBYFD}, \text{LOADMAPBYIDX}, \\ \text{LOADMAPVALBYFD}, \text{LOADMAPVALBYIDX}, \text{LOADFUNCBYIDX}, \text{LOADVARBYBTID}\} \\ S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)] \text{, otherwise} \end{cases}}{}$

(b) Type rules of data handling operations.

MEMLD(dst, src, ioff, size, sign_ext)	$\frac{\begin{array}{c} S.R_{src} = \text{PtrType}(_, _, _) \wedge \text{slot_init}(S, S.R_{src}, \text{ioff}, \text{size}) \\ \boxed{!ptr_slots(S, R_{src}, \text{ioff}, \text{size}), \text{ if size } \neq \text{DW}} \end{array}}{S \rightarrow S[R_{dst} \mapsto \begin{cases} \text{DATA}[R_{src}][b_{src}] & \text{if same_type}(S, S.R_{src}, \text{ioff}, \text{size}) \wedge \text{size} = \text{DW} \\ \text{Scalar}(_, _) & \text{otherwise} \end{cases}]}$
MEMSTX(dst, src, ioff, size)	$\frac{\begin{array}{c} S.R_{dst} = \text{PtrType}(_, _, _) \wedge S.R_{src} \neq \text{Uninit} \wedge \\ \boxed{S.R_{src} \notin \{\text{PtrType}(_, _, _), \text{PtrOrNullType}(_, _, _)\}, \text{ if size } \neq \text{DW}} \\ \boxed{!ptr_slots(S, R_{dst}, \text{ioff}, \text{size}), \text{ if size } \neq \text{DW}} \end{array}}{S \rightarrow S \left[\begin{cases} \forall i \in [b, e), \text{DATA}[R_{dst}][i] \mapsto S.R_{src} & \text{if size} == \text{DW} \\ \forall i \in [b', e'), \text{DATA}[R_{dst}][i] \mapsto \text{Scalar}(_, _) & \text{if size } \neq \text{DW} \wedge ptr_slots(S, R_{dst}, \text{ioff}, \text{size}) \\ \forall i \in [b, e), \text{DATA}[R_{dst}][i] \mapsto \text{Scalar}(_, _) & \text{otherwise} \end{cases} \right]}$
MEMST(dst, src_imm, ioff, size)	$\frac{\begin{array}{c} S.R_{dst} = \text{PtrType}(_, _, _) \wedge \\ \boxed{!ptr_slots(S, R_{dst}, \text{ioff}, \text{size}), \text{ if size } \neq \text{DW}} \end{array}}{S \rightarrow S \left[\begin{cases} \forall i \in [b', e'), \text{DATA}[R_{dst}][i] \mapsto \text{Scalar}(_, _) & \text{if size } \neq \text{DW} \wedge ptr_slots(S, R_{dst}, \text{ioff}, \text{size}) \\ \forall i \in [b, e), \text{DATA}[R_{dst}][i] \mapsto \text{Scalar}(_, _) & \text{otherwise} \end{cases} \right]}$
ATOMICLS(dst, src, ioff, asize, ATOMIC_CMPXCHG)	$\frac{S.R_{dst} = \text{PtrType}(\text{PTR_TO_MAP_VAL}, _, _) \wedge \boxed{S.R_{src} = \text{Scalar}(_, _)} \wedge \boxed{S.R_0 = \text{Scalar}(_, _)}}{S \rightarrow S[R_0 \mapsto \text{Scalar}(_, _)]}$
ATOMICLS(dst, src, ioff, asize, *_FETCH ATOMIC_XCHG)	$\frac{S.R_{dst} = \text{PtrType}(\text{PTR_TO_MAP_VAL}, _, _) \wedge \boxed{S.R_{src} = \text{Scalar}(_, _)}}{S \rightarrow S[R_{src} \mapsto \text{Scalar}(_, _)]}$
ATOMICLS(dst, src, ioff, asize, op) op $\in$ {ATOMIC_ADD, ATOMIC_AND, ATOMIC_OR, ATOMIC_XOR}	$\frac{S.R_{dst} = \text{PtrType}(\text{PTR_TO_MAP_VAL}, _, _) \wedge \boxed{S.R_{src} = \text{Scalar}(_, _)}}{S \rightarrow S}$

(c) Type rules of memory operations. We use the below definition to simplify the above formula:

$$\begin{aligned}
\text{DATA}[R] &\stackrel{\text{def}}{\iff} S.\text{mems}[r2id(R.r)][R.\text{memid}].\text{data} \\
\text{slot_init}(S, R_x, \text{ioff}, \text{size}) &\stackrel{\text{def}}{\iff} \forall R_x.\text{off} + \text{ioff} \leq i < R_x.\text{off} + \text{ioff} + \text{size}, \text{DATA}[R_x][i] \neq \text{Uninit} \\
ptr_slots(S, R_x, \text{ioff}, \text{size}) &\stackrel{\text{def}}{\iff} \forall R_x.\text{off} + \text{ioff} \leq i < R_x.\text{off} + \text{ioff} + \text{size}, \text{DATA}[R_x][i] \in \{\text{PtrType}(_, _, _), \text{PtrOrNullType}(_, _, _)\} \\
b_{src} &\stackrel{\text{def}}{\iff} S.R_{src}.\text{off} + \text{ioff} \quad b \stackrel{\text{def}}{\iff} S.R_{dst}.\text{off} + \text{ioff} \quad b' \stackrel{\text{def}}{\iff} b - (b\%8) \quad e \stackrel{\text{def}}{\iff} b + \text{size_to_nat}(\text{size}) \quad e' \stackrel{\text{def}}{\iff} b' + \text{size_to_nat}(\text{DW})
\end{aligned}$$

CONDJMPREG(dst, src, JEQ64 | JNE64)

$$\frac{\frac{S.R_{dst} = \text{PtrOrNullType}(_, _, _) \wedge S.R_{src} = \text{Scalar}(_, _)}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)] \vee S \rightarrow S[R_{dst} \mapsto \text{PtrType}(_, _, _)]} \quad \frac{S.R_{dst} = \text{Scalar}(_, _) \wedge S.R_{src} = \text{PtrOrNullType}(_, _, _)}{S \rightarrow S[R_{src} \mapsto \text{Scalar}(_, _)] \vee S \rightarrow S[R_{src} \mapsto \text{PtrType}(_, _, _)]} \quad \frac{S.R_{dst} \neq \text{Uninit} \wedge S.R_{src} \neq \text{Uninit} \wedge \left(\boxed{S.R_{dst} = \text{Scalar}(_, _) \wedge S.R_{src} = \text{Scalar}(_, _)} \vee \boxed{\forall T \in \{S.R_{src}, S.R_{dst}\}, T \in \{\text{PtrType}(_, _, _), \text{PtrOrNullType}(_, _, _)\}} \right)}{S \rightarrow S}$$

CONDJMPREG(dst, src, jmpop),
jmpop $\in \{\text{JGT64, JGE64, JSGT64, JSGE64, JLT64, JLE64, JSLT64, JSLE64}\}$

$$\frac{S.R_{dst} \neq \text{Uninit} \wedge S.R_{src} \neq \text{Uninit} \wedge \left(\boxed{\forall T \in \{S.R_{src}, S.R_{dst}\}, T = \text{Scalar}(_, _)} \vee \boxed{\forall T \in \{S.R_{src}, S.R_{dst}\}, T \in \{\text{PtrType}(_, _, _), \text{PtrOrNullType}(_, _, _)\}} \right)}{S \rightarrow S}$$

CONDJMPREG(dst, src, jmpop),
jmpop $\in \{\text{*32, JSET64}\}$

$$\frac{S.R_{dst} \neq \text{Uninit} \wedge S.R_{src} \neq \text{Uninit} \wedge \boxed{\forall T \in \{S.R_{src}, S.R_{dst}\}, T = \text{Scalar}(_, _)}}{S \rightarrow S}$$

CONDJMPIMM(dst, src_imm, JEQ64 | JNE64)

$$\frac{\frac{S.R_{dst} = \text{PtrOrNullType}(_, _, _) \wedge \text{src_imm} = 0}{S \rightarrow S[R_{dst} \mapsto \text{Scalar}(_, _)] \vee S \rightarrow S[R_{dst} \mapsto \text{PtrType}(_, _, _)]} \quad \frac{S.R_{dst} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _)}}{S \rightarrow S}$$

CONDJMPIMM(dst, src_imm, jmpop),
jmpop $\notin \{\text{JEQ64, JNE64}\}$

$$\frac{S.R_{dst} \neq \text{Uninit} \wedge \boxed{S.R_{dst} = \text{Scalar}(_, _)}}{S \rightarrow S}$$

CALL_PSEUDO_FUNC()

$$\frac{}{S \rightarrow S[\forall i \in [6, 9], R'_i \mapsto R_i]}$$

CALL_KFUNCS()
CALL_HELPER()

$$\frac{\forall i \in [1, \text{args}T'_{cnt}], \text{args}T'[i] = S.R_i.T, \text{ where } \text{args}T' \text{ is argument types of a kernel function.}}{S \rightarrow S[R_0 \mapsto \text{ret}T, \forall i \in [1, 5], R_i \mapsto \text{Uninit}]}$$

EXIT

$$\frac{S.R_0 = \text{Scalar}(_, _), \text{ if } S.R_{10}.memid = 0}{S \rightarrow S \left[\begin{array}{l} \forall i \in [1, 5], R_i \mapsto \text{Uninit} \\ \forall i \in [6, 9], R_i \mapsto R'_i, \text{ if } S.R_{10}.memid \neq 0 \\ \forall i \in |\text{DATA}[S.R_{10}]|, \text{DATA}[S.R_{10}][i] \mapsto \text{Uninit} \end{array} \right]}$$

(d) Control flow operations.

Figure 14. The type rules in SPECHECK.